VLM-Rec: A Multimodal Recommendation Cascade and a Stage-Matched Audit of Generative Retrieval

Guangbo Yu

Technical report, November 2026

github.com/guangboyu/multimodal-cascade-recsys

Abstract

We present VLM-Rec, a recommender built end to end on Amazon Reviews 2023 that follows the industrial cascade of retrieval, pre-ranking, ranking, post-processing and online serving. Item representation comes from frozen text and image encoders together with structured item profiles written by a vision-language model, and every design choice is tested by a controlled ablation. On Video_Games (95K users, 25.6K items), a content-only two-tower retriever reaches Recall@100 of 0.214 against 0.129 for an ID-only tower (+66%, and +81% on the cold slice), and removing content features from the ranker costs 0.097 GAUC. A diagnosis of cascade sample-selection bias, with two targeted repairs, lifts cascade NDCG@10 by up to 48%. The full cascade serves at 16 ms p99 on CPU and runs on a catalog eight times larger (208K items) through a configuration change alone, where the headline comparisons replicate. We then use the system as a laboratory for the question a team adopting semantic-ID generative retrieval must answer: which stage loses the recall? Because recommendation catalogs are finite, catalog-wide lazy-exact top-K ranking of an autoregressive retriever is computable, which lets us measure a stage-matched ladder from the dense retriever to the deployed trie-beam system on three catalogs and three seeds. Content quantization is the largest gross entry (58–92% of the Recall@10 gap), trie renormalization is equivalence-negligible, cut-off-matched beam search is secondary but nonzero (5–19%), and on Video_Games the recall difference between parameter-matched contrastive and autoregressive heads changes sign between decoder widths of 3.6M and 13.4M parameters.

1 Introduction

Production recommenders are cascades. A cheap retriever narrows a large catalog to a few hundred candidates, heavier models re-order them, and business rules shape the final slate. Public work concentrates on the ranking model, yet every stage of a cascade reads the same item representation, and that representation decides what the system can do for items with little interaction history. VLM-Rec puts multimodal item understanding at the center of a complete cascade and measures, stage by stage, what each component contributes.

The second half of the report takes up a question this kind of system raises directly. Generative retrieval with semantic IDs, in the style of TIGER [1], replaces the nearest-neighbour index with a decoder that writes the next item's codes. Our own TIGER-style model reaches half of the two-tower's

Recall@10 with no index at all (Section 4.5). A team deciding whether to migrate needs to know which stage gave up the other half, and an end-to-end comparison cannot say, because it changes the tokenizer, the encoder, the training objective and the decoder together. The stage-level measurement is available, and Section 5 takes it.

The report makes four contributions.

1. A complete, configuration-driven cascade: two-tower retrieval over FAISS with a co-visitation channel, a distilled pre-ranker, a DIN + DCN-v2 + MMoE ranker, MMR and DPP post-processing, and a FastAPI service that answers in 16 ms p99 on CPU and scales from 25.6K to 208K items by editing one file (Section 3, Section 4.7).
2. Controlled measurements of what multimodal content is worth at each stage, including VLM-generated item profiles and RQ-VAE semantic IDs, replicated on a second category (Section 4).
3. A diagnosis of cascade inconsistency that separates two mechanisms, label poisoning by held-out positives and a missing cross-stage score, and repairs both (Section 4.3).
4. A catalog-wide lazy-exact top-K instrument and a stage-matched loss ledger for the dense-to-generative retrieval gap on three catalogs, with hierarchical bootstrap confidence intervals on every entry (Section 5).

2 Data and item representation

2.1 Catalogs and splits

We use Amazon Reviews 2023 [2]. Video_Games is the development catalog: its listings pair distinctive box art with rich text, and its size permits fast iteration on one GPU. Beauty_and_Personal_Care is the scale catalog, eight times larger in items and 7.7 times sparser. The loss-ledger study adds the Amazon-2014 Beauty split used by TIGER, whose user, item and interaction counts our preprocessing reproduces exactly. Every split is temporal leave-last-out: each user's last interaction is the test target and the second-to-last is the validation target. Table 1 summarizes the catalogs.

Throughout the system results, the *cold slice* is the set of test users whose target item falls in the bottom quartile of training popularity. It isolates the long tail, where collaborative signal is weakest and content has to carry the prediction.

Table 1. Catalogs after 5-core filtering. Video_Games splits into 625,062 / 94,762 / 94,762 train, validation and test interactions, and 99.98% of its items have an image. The two 2023 catalogs come from Amazon Reviews 2023 and are used throughout; Amazon-2014 Beauty uses the TIGER split and enters only the loss ledger of Section 5.

Catalog	Raw reviews	Users	Items	Interactions	Density
Video_Games	4.62M	94,762	25,612	814,586	0.034%
Beauty_and_Personal_Care	23.9M	729,576	207,649	6,624,441	0.0044%
Amazon-2014 Beauty	–	22,363	12,101	198,502	0.073%

2.2 Content features

Each item is described by three frozen embeddings, computed once and aligned by item index, so row i of every matrix refers to the same product. Listing text (title, description and feature bullets) is encoded with all-MiniLM-L6-v2 [3] into 384 dimensions. The main product image is encoded with CLIP ViT-B/32 [4] into 512 dimensions, with a zero vector and a flag for the few items without one. A structured profile written by a vision-language model is encoded into a further 384 dimensions. Concatenated, these form a 1,281-dimensional content vector that every stage of the cascade shares.

Keeping the encoders frozen is deliberate. It makes every downstream ablation cheap, and it attributes a gain to the information an encoder supplies rather than to a fine-tuning budget.

2.3 VLM item profiles

Qwen2.5-VL-7B [5] reads each item's image and title and writes a JSON profile with a fixed schema: refined category, sub-genre, visual style, key attributes, target audience, tone, quality cues and a one-line summary. A deterministic template turns the profile into text, which passes through the same MiniLM encoder as the raw listing, so the profile block differs from the text block only in what the VLM said. Generation is batched, checkpointed in shards with atomic writes, and resumable after interruption. The full Video_Games catalog took 243 minutes on one RTX 4090 and produced valid JSON for 100% of items; the Beauty catalog took 16.8 hours with Qwen2.5-VL-3B at 99.76% validity.

Throughput and validity both depend on details of batched generation. Decoder-only models must be padded on the left: with the tokenizer's default right padding, every row but the longest in a batch continues after pad tokens, and validity fell to 3 of 24 rows. Batch size must also be re-tuned per model size. The 3B model at the 7B model's batch size left the GPU 61% utilized; a batch of 24 brought utilization to 100% and throughput up 2.5 times.

3 System

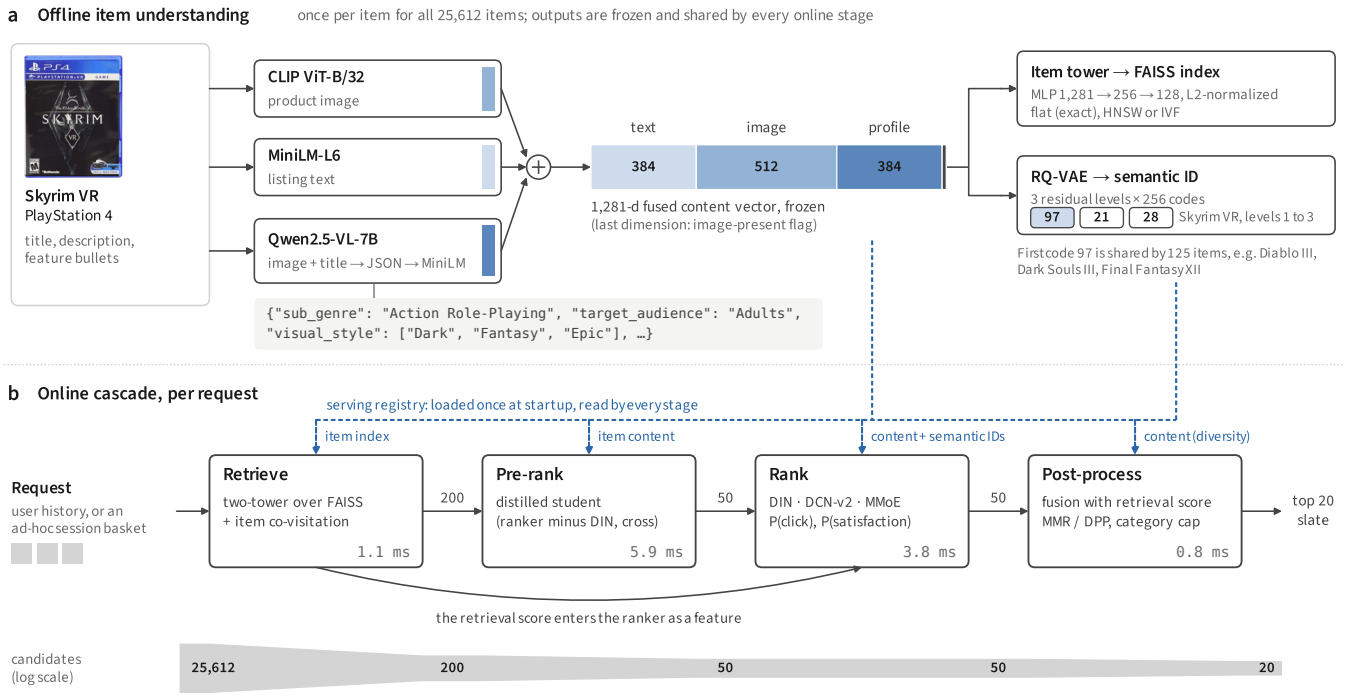


Figure 1. System overview. **a**, Offline, three frozen encoders read each item once, shown here for a real catalog item with an excerpt of the JSON profile Qwen2.5-VL wrote for it. Their outputs concatenate into the 1,281-d content vector, drawn to scale by block. The item tower turns that vector into the FAISS index, and the RQ-VAE turns it into a three-level semantic ID; the example item's first code is shared by 125 items, whose most popular members are fantasy role-playing games. **b**, Online, a request passes through four stages and returns 20 items by default. The ribbon shows candidates per request on a log scale, and stage latencies are CPU means from Table 8. Every stage reads the same registry of offline artifacts (dashed), and the retrieval score travels with each candidate into the ranker.

3.1 Retrieval

The retriever is a two-tower model. The item tower maps the content vector through a two-layer MLP (1,281 → 256 → 128) and normalizes the output. The user tower applies an MLP of the same shape to the mean content vector of the user's training history. Neither tower reads a user ID, so one model scores a logged user and a brand-new session alike. Training uses an in-batch sampled softmax with temperature τ and the log-frequency correction of Yi et al. [6], which removes the penalty that popular items otherwise receive for appearing more often as in-batch negatives:

$$\mathcal{L} = -\log \frac{\exp(s(u, i^+)/\tau - \log q_{i^+})}{\sum_{j \in B} \exp(s(u, j)/\tau - \log q_j)} \tag{1}$$

where s is cosine similarity, B is the batch and q_j is the sampling frequency of item j. For ablation the tower has two further modes: *hybrid* adds learned item-ID, price-bucket and category embeddings to the content input, and *ID-only* replaces content with learned user and item vectors. FAISS [7] serves the item embeddings from a flat, HNSW [8] or IVF index. A second candidate source counts item-to-

item co-visitation in training histories, damped by popularity. Retrieval passes 200 candidates downstream.

3.2 Ranking

The ranker is where the modeling budget goes. A DIN [9] target-attention block weights the user's behaviour sequence by its relevance to the candidate, with the candidate masked out of its own history so a positive cannot leak through it. A DCN-v2 [10] cross network models explicit feature interactions. An MMoE [11] layer shares experts between two task heads, P(click) and P(satisfaction), where satisfaction means a rating of four or more. Structured features (price, category leaf, item ID) enter as quantile buckets with learned embeddings. Two inputs are switched by configuration: the candidate's retrieval score $u \cdot i$, the dot product of the user and item vectors (Section 4.3) and semantic-ID code embeddings (Section 3.4). A small metadata file saved with each checkpoint records which inputs it uses, and the serving path builds the model from that file.

3.3 Pre-ranking and post-processing

Running the full ranker over 200 candidates spends most of its work on items that never reach the user. A pre-ranker with the ranker's architecture minus the DIN and cross blocks is distilled from the ranker's click logits and cuts 200 candidates to 50; its top 50 retains 79% of the full ranker's top 10. Post-processing multiplies the two task scores, P(click)·P(sat), blends in the retrieval score with a weight α tuned on validation, then trades relevance for diversity with maximal marginal relevance [12] or greedy MAP inference for a determinantal point process [13] over item content embeddings, and finally applies a per-category cap.

3.4 Semantic IDs

An RQ-VAE [14] quantizes each content vector into three levels of 256 codes, so items with similar content share code prefixes and a new item receives a meaningful identifier from its content alone. Residual quantizers are prone to codebook collapse, so training initializes codebooks by k-means on first-batch residuals, updates them by exponential moving average, re-seeds dead codes, and fails any run whose per-level utilization drops below 60%. The codes serve three purposes: they replace the learned item ID in the two-tower, they enter the ranker as features, and they form the vocabulary of a TIGER-style generative retriever. That retriever is a small decoder-only transformer that reads the user's history as code tokens and generates the next item's codes by beam search constrained to a trie of real items, so every output is a valid item and no ANN index is involved.

3.5 Serving and operations

At startup a registry loads every artifact into memory on CPU: both towers, the FAISS index, the pre-ranker, the ranker and the item feature tables. Each request then runs the four stages in sequence and records the latency of each. A FastAPI application exposes the cascade at `GET /recommend` for logged users and at `POST /recommend/session` for an ad-hoc basket of items; because no stage reads a user ID, a new session receives a personalized feed with no retraining. An explain mode returns the candidates and scores after every stage. The ranker exports to ONNX with its feature tables baked in and matches PyTorch to a maximum absolute difference of 2.4×10^{-6}.

Around the service sit a Docker Compose deployment with a Streamlit demo, Prometheus metrics with per-stage latency histograms, MLflow run tracking, and continuous integration that runs ruff and a pure-logic test suite on every push. Every stage is one CLI command driven by a single YAML file, and command-line overrides are logged with each run. Table A1 lists the key parameters of every stage.

4 Results: the cascade

All results in this section are on the test split under temporal leave-last-out, with model variants chosen on the validation split. Unless stated otherwise the catalog is Video_Games and the item content is the fused 1,281-d vector. R@K denotes Recall@K, the fraction of test users whose held-out next item appears in the top K.

4.1 Retrieval

Table 2. Candidate sources on Video_Games. Recall@K of the held-out next item; the cold column restricts to targets in the bottom popularity quartile.

Source	R@10	R@100	R@500	R@100, cold
Two-tower, content	0.063	0.214	0.408	0.044
Two-tower, hybrid	0.063	0.216	0.415	0.041
Two-tower, ID-only	0.036	0.129	0.263	0.024
Item-to-item co-visitation	0.038	0.116	0.170	0.065
Popularity	0.025	0.097	0.233	0.000

Content beats identity. The content tower reaches Recall@100 of 0.214 against 0.129 for the ID-only tower, a 66% gain that grows to 81% on the cold slice, and it does so without per-user or per-item embedding tables. The ID-only tower fits its training data more tightly and generalizes worse, the expected behaviour of free per-item vectors on sparse data. Adding ID, price and category embeddings to the content tower (hybrid) leaves overall recall where it was and lowers cold-slice recall from 0.044 to 0.041, so structured-feature work belongs in the ranker.

The sources complement each other. Co-visitation is the weakest source overall and the strongest on the cold slice (0.065), while popularity retrieves nothing there; together these numbers make the case for blending a learned retriever with a co-visitation channel. An HNSW index recovers 81% of the exact top 100 at its default search depth.

4.2 Ranking

Table 3. Ranker ablation on Video_Games. GAUC is the per-user AUC of click prediction, averaged over users, with ties scored at mid-rank.

Variant	GAUC	GAUC, cold
Full (DIN + DCN-v2 + MMoE, content features)	0.858	0.701
without content features	0.761	0.347
without DIN	0.852	0.692
without DCN-v2 cross network	0.852	0.701
single task (no MMoE)	0.856	0.703

Content is the dominant ranking input as well. Removing it lowers GAUC from 0.858 to 0.761 and drops cold-slice GAUC to 0.347, below the 0.5 of a random ranker: without content, a rarely seen item gives the model nothing to separate it from noise. DIN and the DCN-v2 cross network each add 0.006 GAUC. The MMoE head leaves click GAUC unchanged (0.856 single-task against 0.858) and adds the satisfaction objective without cost to click quality.

4.3 Cascade consistency

Re-ranking retrieved candidates exposes a consistency problem that offline ranking metrics do not show. Table 4 isolates its mechanisms by measuring the NDCG@10 of each ranker's order over the retriever's top 200. A ranker trained on random negatives scores 0.858 GAUC yet orders these candidates at 0.081, because random negatives are far easier than the candidates retrieval surfaces; this is sample-selection bias between stages. Mining negatives from the candidate set exposes a second mechanism, label poisoning. The candidate file masked only training items, and because retrieval works, a user's held-out positive is usually among the candidates, so it was drawn as a negative and labelled click = 0 (0.051). Rejection-resampling every negative that collides with the row positive or with the user's validation and test items repairs the pool (0.060).

Table 4. Cascade consistency on Video_Games: NDCG@10 of each ranker's order over the retriever's top 200, with the text+image features (897-d) of the original run. GAUC is measured against random negatives, a task the listwise variants no longer optimize.

Ranker	Cascade NDCG@10	GAUC
Retrieval order (reference)	0.109	–
Random negatives	0.081	0.858
Hard negatives, poisoned pool	0.051	–
Hard negatives, clean pool	0.060	0.791
+ retrieval-score feature	0.075	0.825
+ listwise softmax loss	0.064	0.647
+ serving-like slate (75% hard, from the top 50)	0.032	0.636

The third mechanism is a missing cross-stage signal. Without the retrieval score the ranker can only replace the retriever's opinion, never refine it. Adding the raw two-tower score $u \cdot i$ as an input, a quantity defined for every user-item pair and identical to the FAISS score at serving time, lifts cascade NDCG@10 to 0.075, 48% above the poisoned pool. How the feature is built matters. A binary was-retrieved flag looks equivalent but would encode the label in reverse: training positives are masked out of the candidate file, so every positive would carry flag 0 and most hard negatives flag 1. The same residual bias appears when hard negatives are concentrated in the top 50, the slate most correlated with candidate membership, which drops cascade NDCG@10 to 0.032. Mixing easy and hard negatives, as reported for embedding-based retrieval at Facebook [15], is the stable regime.

Two properties of the evaluation govern how these numbers are used. The candidates were selected by the retriever's own similarity, which gives the retriever's order a home-field advantage on this metric, and validation-tuned fusion settles at α between 0.75 and 1.0. The served path therefore ranks by the fused score, which on validation is never worse than retrieval order because $\alpha = 1$ is in the grid, and the comparison free of the advantage is an online one. The winning variant also depends on the feature set: the text+image run selects the score-feature ranker, the fused-feature run selects the clean hard-negative ranker (0.063, +23% over its poisoned baseline), and Beauty selects the listwise score-feature ranker. Selection is therefore automated on validation, and the chosen checkpoint is saved with a metadata file that describes its inputs, so serving follows every change without code edits.

4.4 VLM item profiles

Table 5. Content-source ablation on Video_Games under identical retrieval and ranking recipes. Dimensions include the image-present flag where the image block is used.

Sources	Dim	R@100	R@100, cold	GAUC	GAUC, cold
text + image	897	0.2104	0.0440	0.8586	0.7033
image	513	0.1924	0.0302	0.8495	0.6820
VLM profile	385	0.1877	0.0287	0.8444	0.6640
image + VLM profile	897	0.2126	0.0393	0.8586	0.7046
text + image + VLM profile	1281	0.2135	0.0436	0.8577	0.6994

The full combination gives the best retrieval (Recall@100 0.2135, +1.5% over text and image) and is the production feature set. Two comparisons show what the profile carries. On its own, the 385-d profile block nearly matches the 513-d image block (0.188 against 0.192), so one short text embedding of the profile captures most of what the image embedding provides. Paired with the image at equal width, the profile matches the raw listing text (0.2126 against 0.2104), so it can stand in for the listing. Video_Games listings are text-rich, and much of what the profile says is already in the raw text; on the sparser Beauty catalog the lift doubles to +3.1% (0.0859 against 0.0833). Profiles leave the cold slice unchanged on both catalogs, and the next subsection shows that the identifier pathway is what moves it.

4.5 Semantic IDs and generative retrieval

The RQ-VAE trains in about 16 seconds on the fused vectors and uses its codebooks fully: 100% utilization on all three levels and perplexities of 208, 249 and 252 out of 256. Of the 25,612 items, 9.6% share a full code tuple with another item, and the generative retriever resolves these collisions by within-group popularity.

Table 6. Semantic IDs on Video_Games. Top: item representation inside the two-tower retriever. Bottom: semantic-ID code embeddings as extra ranker inputs.

Two-tower item representation	R@100	R@100, cold
learned item ID	0.1288	0.0241
semantic ID (codes)	0.0835	0.0353
content	0.2135	0.0436
content + semantic ID	0.2109	0.0416
Ranker	**GAUC**	**GAUC, cold**
base	0.8580	0.6993
+ semantic-ID features	0.8610	0.7076

Semantic IDs act exactly where learned IDs are weakest. In place of the learned item vector they lift cold-slice Recall@100 by 46% (0.0241 → 0.0353), because a new item's codes are already trained by every item that shares them, and the gain replicates on Beauty at +47%. Warm items still prefer a free per-item vector, so overall recall favors learned IDs (0.129 against 0.084); the two representations divide the catalog by popularity. As ranker features the codes add 0.003 GAUC, and 0.008 on the cold slice, even on top of full content. Appending codes to content in the retriever leaves recall unchanged, since codes derived from content carry nothing the content lacks; their role is a compact, content-grounded identifier.

The TIGER-style generative retriever reaches Recall@10 of 0.031 on 10,000 sampled users, half of the two-tower's 0.063, with one million parameters, a 769-token vocabulary, 48 seconds of training and no ANN index. That gap is the subject of Section 5, which measures where a generative stack loses recall relative to a dense one.

4.6 Diversity and latency

Post-processing exposes a smooth relevance-diversity frontier (Table 7). Lowering the MMR weight λ, or switching to DPP, buys intra-list diversity with relevance. DPP gives the most diverse lists, 22% above pure relevance at 42% lower NDCG. Where to sit on this frontier is a product decision, and the API accepts the diversity method per request.

Table 7. Relevance and intra-list diversity of the final slate over the pre-ranked top 50.

Method	NDCG	Intra-list diversity
MMR, $\lambda = 1.0$ (pure relevance)	0.0314	0.316
MMR, $\lambda = 0.7$	0.0282	0.352
MMR, $\lambda = 0.5$	0.0236	0.367
DPP (greedy MAP)	0.0181	0.385

On CPU the cascade answers in 11 ms at the median and 16 ms at p99 (Table 8), well inside a 100 ms budget. The pre-ranker is the most expensive stage because it scores all 200 candidates, while the heavy ranker sees only 50. With the fused 1,281-d features the cascade measures 12 ms p50 and 16.4 ms p99 over a 300-request load test, and the ONNX Runtime path returns item-identical results.

Table 8. Serving latency on CPU, warm, over a 200-user sample with the text+image stack and an exact (flat) FAISS index.

Stage	Mean latency
Retrieve (FAISS, seen items masked)	1.1 ms
Pre-rank (200 → 50)	5.9 ms
Rank (DIN + DCN-v2 + MMoE)	3.8 ms
Post-process (fusion, MMR or DPP)	0.8 ms
End to end	p50 11 · p95 14 · p99 16 ms

4.7 Replication at scale

The Beauty run reuses every line of code; one configuration file points the pipeline at a category-scoped data directory. At 729,576 users and 207,649 items, the 7.4 GB image download takes 25.7 minutes with 32 workers, CLIP encoding takes 4 minutes with DataLoader workers, and the serving registry loads in 60 seconds into 11 GB of memory. With an HNSW index the cascade serves at 8.6 ms p50 and 10.1 ms p99. Table 9 lists the comparisons repeated on both catalogs; absolute recall is lower on Beauty because it is 7.7 times sparser, so the within-catalog contrasts carry the signal.

Table 9. The same comparisons on both catalogs.

Comparison	Video_Games	Beauty
Content vs. ID-only two-tower, R@100	+66%	+75%
Ranker without content, GAUC change	−0.097	−0.06
Ranker without content, cold GAUC	0.70 → 0.35	0.67 → 0.40
VLM profile lift, R@100	+1.5%	+3.1%
Semantic vs. learned item ID, cold R@100	+46%	+47%
Ranker + semantic-ID features, GAUC / cold GAUC	+0.003 / +0.008	+0.002 / +0.005
RQ-VAE utilization; perplexity range	100%; 208–252	100%; 219–253
Strongest cold-slice source	co-visitation	co-visitation
Serving p50 / p99	12 / 16 ms (flat)	8.6 / 10.1 ms (HNSW)

5 Where does generative retrieval lose recall?

A generative retriever differs from a dense retriever in several places at once. Item content is quantized into discrete codes, the encoder and the construction of training examples change, the scoring head and its objective change, and candidates come from a trie-constrained beam instead of a nearest-neighbour lookup. End-to-end comparisons vary all of these together, so the contribution of each stage is not isolated, and the decoder tends to take the blame because it is the easiest stage to point at. This section measures each stage on its own.

5.1 Exact top-K ranking for a generative retriever

Let X be the item catalog. A tokenizer assigns each item x a code tuple $(c_1, \ldots, c_L)$, and given a user history h an autoregressive model scores the item by

$$s(x \mid h) = \sum_{\ell=1}^{L} \log p(c_\ell \mid c_{<\ell}, h) \;\leq\; 0\,. \tag{2}$$

Every term is non-positive, so the score of a code prefix is an upper bound on the score of every item that starts with it. A best-first search over the code tree, which always expands the highest-scoring prefix, can stop as soon as the K-th complete item outscores the best remaining prefix. It then returns exactly the top K items, with no beam width and no approximation, while touching a small fraction

of a finite catalog. We compute it in two variants: with the model's raw probabilities, and with each level's probabilities renormalized over the codes that lead to a real item (trie renormalization). The search agrees with brute-force enumeration on small test catalogs. It makes generative retrieval without approximate decoding an observable system, which separates what the training objective costs from what the decoder costs.

5.2 A ladder from dense to generative retrieval

Table 10 lists seven retrievers that walk from a dense retriever to a deployed generative one. Each differs from the one before it in one stage, so the difference in Recall@10 between neighbours is one entry of a loss ledger, and the entries add up to the total gap between the first and the last system. One step, from code embeddings to the sequential contrastive model, changes the encoder family and the construction of training examples together, and we report it as one joint entry.

Table 10. The seven systems of the ladder, in order. The first three share the two-tower architecture of Section 3.1 and differ only in the item input; the next two share one transformer encoder over code tokens and differ only in the head.

System	What it is	What changes from the previous system
Dense retriever	two-tower over the frozen content vector	–
Reconstructed content	the same two-tower, trained on each item's RQ-VAE reconstruction	quantization
Code embeddings	the same two-tower, whose item input is the concatenated embeddings of the item's three codes	discrete codes replace the reconstruction
Sequential contrastive	a transformer reads the history as code tokens; its output is matched to item vectors with an in-batch softmax	encoder family and training examples (joint)
Exact autoregressive	the same transformer trained to generate the next item's codes; every item ranked exactly (Section 5.1)	scoring head and objective
+ trie renormalization	as above, with probabilities renormalized over codes that lead to real items	trie renormalization
Trie beam search	the deployed decoder: beam search of width 10 inside the trie, filled to K valid items	approximate decoding

The matched pair. The step from the sequential contrastive model to the exact autoregressive model carries the main comparison, so the two systems are matched rather than merely comparable. They share the same encoder architecture and initialization, consume byte-identical chronological next-item examples in the same seeded order, have the same number of parameters to within 0.02% (999,168 against 999,041), and receive identical validation-selected training budgets. Only the scoring head and its training objective differ, and we call this entry the head-and-objective step.

Protocol. Every system is evaluated under one protocol: a shared candidate set per user, a global tie rule (score, then popularity, then item index), and beams filled so that every user receives at least min(K, 200) valid items, with zero shortfalls across the nine main ladders (three catalogs, three seeds each). Every entry carries a hierarchical seed × user bootstrap confidence interval: seeds are resampled, user rows are resampled jointly across the drawn seeds, and both sources of uncertainty enter the interval. Equivalence uses two one-sided tests (TOST) at a margin of 10% of the local gap. Designs, margins and the stability rule of Section 5.6 were registered before the runs. Item content is the fused vector on the 2023 catalogs and text alone on Amazon-2014, and Table A1 lists the training parameters.

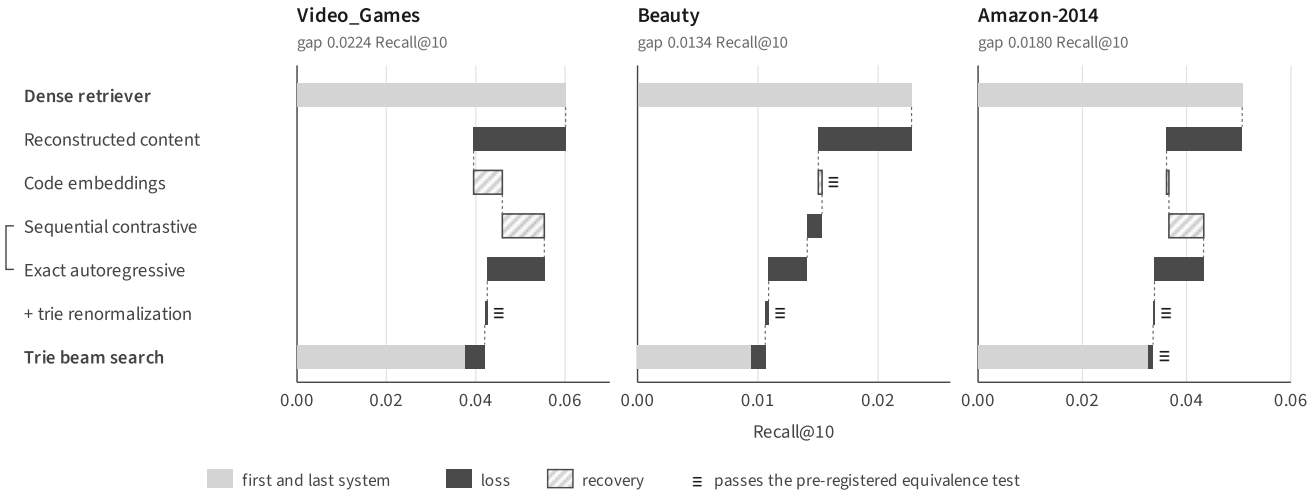


Figure 2. The loss ledger in absolute Recall@10, averaged over three seeds. Each row is one system of Table 10. Between the first and the last row, each bar is the change into that system: solid for a loss, hatched for a recovery. The bracket marks the matched pair. Hover a bar for its value and its share of the gap.

Table 11. The loss ledger at Recall@10. Each row is one step of the ladder, as a share of the total gap, with hierarchical seed × user bootstrap 95% intervals over three seeds. Positive entries are losses and negative entries are recoveries; ≡ marks entries that pass the pre-registered equivalence test. The last row is the span of the share attributed to the autoregressive model across the three comparison systems of Section 5.6.

Step	Video_Games	Beauty	Amazon-2014
Total gap (Recall@10)	0.0224	0.0134	0.0180
Quantization	91.9% [79.4, 106.4]	58.2% [48.6, 67.6]	80.5% [68.7, 94.2]
Discrete codes	−28.6% [−42.0, −16.3]	−2.4% ≡ [−11.2, 6.3]	−2.5% [−15.5, 11.0]
Sequential encoder and examples (joint)	−41.9% [−56.6, −30.8]	9.2% [0.9, 17.4]	−37.0% [−54.7, −22.3]
Head and objective (matched pair)	56.9% [46.4, 68.5]	24.0% [13.2, 36.2]	52.3% [37.6, 67.0]
Trie renormalization	2.5% ≡ [0.2, 5.3]	2.1% ≡ [−0.3, 4.9]	1.4% ≡ [−1.5, 4.1]
Beam search (width 10)	19.2% [14.0, 24.8]	8.9% [1.8, 16.2]	5.3% ≡ [0.7, 10.1]
Span across comparison systems	70.5 pts [56.3, 88.3]	9.2 pts [2.9, 18.4]	39.5 pts [24.4, 60.9]

5.3 Decoding is secondary but nonzero

Trie renormalization passes the equivalence test on all three catalogs, at 2.5%, 2.1% and 1.4% of the respective gaps, and it keeps passing at $K = 50$ and $K = 100$. Beam search at the matching cutoff accounts for 19.2% of the Video_Games gap at Recall@10, 8.9% on Beauty and 5.3% on Amazon-2014, and its share falls as the cutoff grows, to 2.7%, 1.2% and 0.4% at $K = 100$. The stage that attracts most of the decoding effort in the generative-retrieval literature is therefore not where most of the recall goes. It is not free either: a fifth of the Video_Games gap at the cutoff a product would deploy is large enough to tune for.

5.4 Quantization dominates and responds to a controlled dose

Quantization, the step from the dense retriever to reconstructed content, is the largest entry on every catalog: 91.9% of the gap on Video_Games, 58.2% on Beauty and 80.5% on Amazon-2014. To test whether that share reflects the tokenizer rather than where the ladder starts, we intervene on it. Holding the code structure at three levels of 256 codes, we train the tokenizer to three operating points and re-run the ladder at each. Reconstruction error falls monotonically, and the quantization entry falls with it, from 0.0331 to 0.0240 to 0.0206 Recall@10 on Video_Games, with separated endpoint confidence intervals. On Video_Games the discrete codes also win back part of what reconstruction lost: code embeddings retrieve better than the vectors the RQ-VAE reconstructs from the same codes, a recovery of 28.6% of the gap. That recovery shrinks as the tokenizer improves (0.0167 → 0.0097 → 0.0064 Recall@10).

5.5 The head-and-objective step depends on capacity

At the matched one-million-parameter point, the head-and-objective step is a loss on all three catalogs, with intervals excluding zero: 56.9% of the gap on Video_Games, 24.0% on Beauty and 52.3% on Amazon-2014. This is what a team comparing lightweight stacks at this scale would measure, and it describes an operating point rather than a constant. We retrain both heads at each decoder width under a frozen recipe, with learning rate and temperature fixed at their width-128 values and with identical examples and validation selection. On Video_Games the paired difference declines monotonically and changes sign (Figure 3): +0.01725 at width 64 (0.4M parameters), +0.01275 at width 128 (1.0M, the main ladder), +0.00645 at width 256 (3.6M) and −0.00807 [−0.01133, −0.00502] at width 512 (13.4M). The sign change falls between widths 256 and 512, and at width 512 the autoregressive head beats its parameter-matched contrastive twin in every seed.

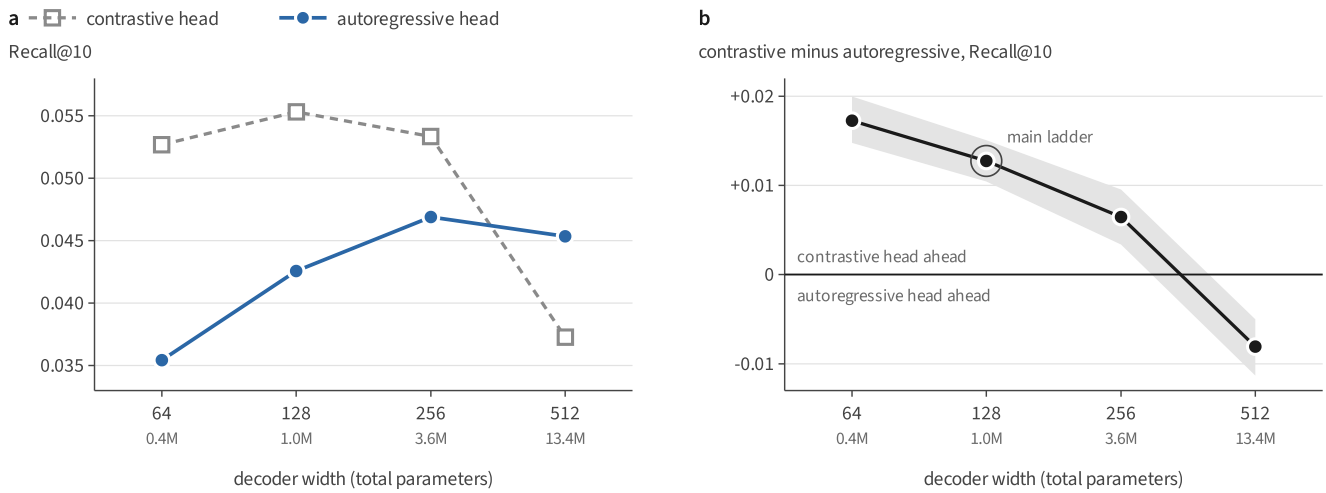


Figure 3. The ordering of the matched heads depends on capacity (Video_Games, frozen recipe). **a**, Recall@10 of the contrastive head and the autoregressive head, both retrained at each width. **b**, Their paired difference, contrastive minus autoregressive, with its hierarchical 95% bootstrap interval; positive values favor the contrastive head. Width 128 is the main ladder.

Both heads move. The autoregressive head improves steadily with width, while the contrastive head peaks at width 256 and falls back. The autoregressive head's disadvantage at small width is a property of the tested regime, not of the objective as such.

Where the loss sits. At Recall@10 the head-and-objective loss concentrates in the upper popularity deciles and peaks in the most popular decile on all three catalogs (interaction between the most popular decile and the rest of +0.062, +0.040 and +0.057, with intervals excluding zero), whereas the quantization loss spreads across the more popular half of the catalog. One candidate explanation is that the autoregressive head mis-calibrates item popularity, so we ran ten training-time logit-adjustment runs on Video_Games with marginal and conditional-prefix priors at $|\tau| \leq 0.5$. None produced a statistically resolved Recall@10 improvement, and every run at $|\tau| = 0.5$ lowered Recall@10, by 0.0025 to 0.0077. Within this family of priors, the loss is not a popularity-calibration artifact.

5.6 Attribution depends on the comparison system

Every entry above is measured directly. A statement such as "switching to an autoregressive model costs *X*% of the gap" needs one more choice: the non-autoregressive system to compare against. Three systems on the ladder are reasonable choices: the sequential contrastive model (our primary choice, the matched twin), the code-embedding two-tower and the reconstructed-content two-tower. Figure 4 shows the share of the gap attributed to the autoregressive model under each. On Video_Games it runs from +56.9% against the sequential contrastive model to +15.0% against code embeddings and −13.6% against reconstructed content, a span of 70.5 percentage points [56.3, 88.3]; none of 2,000 bootstrap replicates falls within the pre-registered 15-point stability rule. The same measured systems thus support both "the autoregressive model costs 57% of the gap" and "the autoregressive model recovers 14% of it". Amazon-2014 spans 39.5 points, with 3 of 2,000 replicates within the rule. Beauty spans 9.2 points and satisfies the rule on its point estimate, with bootstrap support of 1,749 of 2,000.

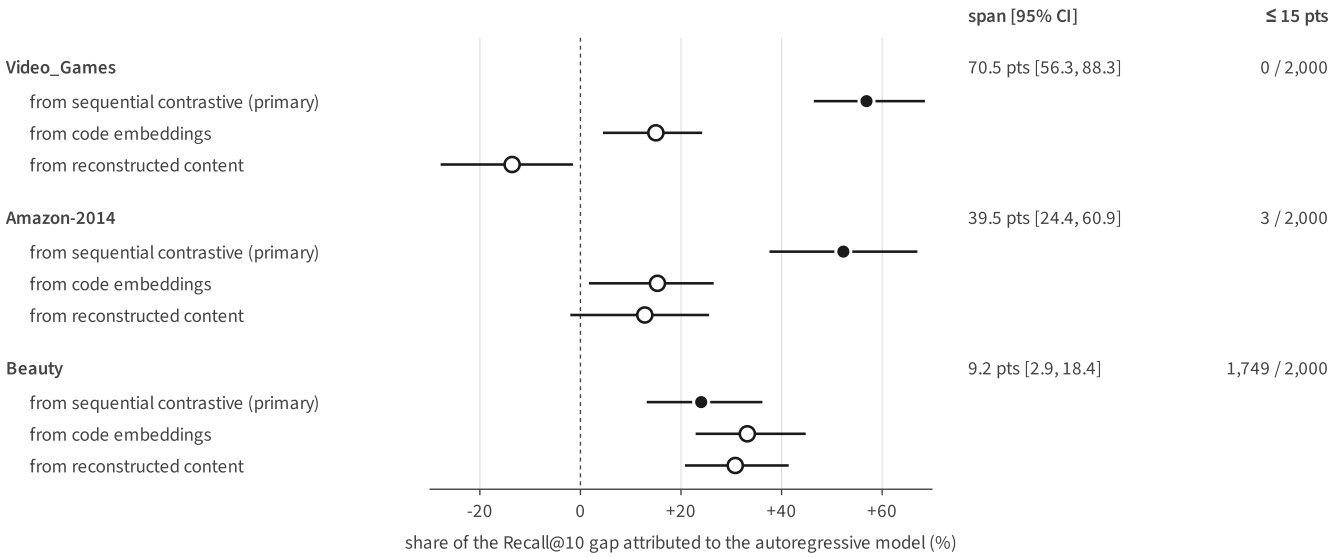


Figure 4. Share of the Recall@10 gap attributed to the autoregressive model when it is compared against each of three systems, with hierarchical 95% bootstrap intervals (filled marker: primary comparison). The span is the range across comparisons; the count is the number of 2,000 bootstrap replicates whose span falls within the pre-registered 15-point rule.

The cause is the shape of the ladder. It contains measured recoveries (discrete codes at −28.6% and the joint encoder-and-examples step at −41.9% on Video_Games), so comparisons that start at different systems accumulate different amounts before they reach the autoregressive model. The same behaviour appears one level down. Swapping heads on a frozen encoder splits the head-and-objective step into a representation part and a scoring part, and the split depends on the direction of the swap on both catalogs where both directions were run. On Video_Games, a contrastive head trained on the frozen autoregressive encoder makes the step mostly representation (0.0092 against 0.0036), while the reverse swap assigns 0.0181 to the head. We therefore report each entry with its comparison system and confidence interval, together with the span across reasonable comparisons as a sensitivity check. The span costs nothing beyond quantities a ledger already computes, and it tells a reader whether a headline attribution would survive a different, equally defensible choice.

5.7 Implications and scope

For practice. For a team weighing a migration, the ledger reorders the work queue. The largest entries are the tokenizer and the head-and-objective step, while beam search stays secondary at every cutoff measured, so tokenizer training and the discrete interface repay effort sooner than wider beams. Because the ordering of the matched heads reverses with decoder width on the catalog and recipe tested, contrastive and autoregressive stacks should be compared at matched capacity before an ordering is assumed. And because attribution depends on the comparison system, a reported attribution should name its comparison system and give the span across the alternatives.

Scope. Both ends of the ladder are lightweight by construction, matched at about one million parameters; this is what makes a stage-matched ledger possible, and it is what the entries describe. The capacity crossover is demonstrated on Video_Games under a frozen recipe at this data scale, and it establishes that the ordering is not invariant rather than a law every catalog follows. The ledger reports contrasts labeled by their comparison system and is not a unique causal decomposition.

6 Engineering notes

Several failure modes surfaced while the system was built, and each is now guarded in code or configuration. Table 12 lists those with the widest relevance beyond this project; `docs/PITFALLS.md` in the repository records the rest with root causes.

Table 12. Failure modes and their guards.

Symptom	Cause	Guard
Training ran on CPU with no error	default PyTorch wheel built for CUDA 13.0 on a CUDA 12.8 driver	pin CUDA 12.8 wheels; assert GPU at startup
Installing vLLM broke the `transformers` import for the whole environment	the wheel links the CUDA 13 runtime and ships an incompatible audio library	run generation on the `transformers` backend
Batched VLM output valid for one row per batch	right padding in decoder-only generation	left padding (3/24 → 24/24 valid)
Smaller VLM no faster than the larger one	batch size tuned for the larger model; 61% GPU utilization	tune batch size per model; 100% utilization
Hard negatives made the cascade worse	held-out positives sampled as negatives	resample against validation and test items
A was-retrieved flag would leak labels	training positives are removed from the candidate file	use the continuous retrieval score
Serving crashed on the 208K-item catalog	HNSW returns −1 when its search depth cannot supply k results	search depth $\geq k$; drop −1 results
Image cache could pair items with the wrong pictures	cache keyed on an item number that changes with every rebuild	key on Amazon's stable product ID

Symptom	Cause	Guard
A whole source package was missing from git	an ignore rule meant for the data folder also matched a source folder with the same name	anchor the rule to the repository root

7 Takeaways

1. Item representation sets the ceiling of every stage. Content adds 66% to retrieval recall and 0.097 to ranking GAUC, and without it the ranker falls below chance on the cold slice.
2. Content-derived signals divide the catalog. VLM profiles capture most of the multimodal signal in a short embedding and their lift grows with sparsity (+1.5% to +3.1%); semantic IDs carry the cold slice, +46% and +47% on the two catalogs.
3. Cascade consistency is a data problem before it is a model problem. A clean negative pool and a continuous cross-stage score recover up to 48% of cascade NDCG@10, and a feature has to be judged by how it is built at training time, not by what it means at serving time.
4. Model choices should be selected, not fixed. Three runs produced three different winning cascade variants, and validation selection, with a metadata file saved beside each checkpoint, kept serving consistent through each change.
5. In the generative stack, the largest losses sit upstream of the decoder. Quantization is the largest entry on every catalog, trie renormalization is equivalence-negligible, and beam search is secondary but nonzero.
6. The ordering of contrastive and autoregressive heads depends on capacity, and gap attribution depends on the comparison system. Compare at matched capacity, and report the comparison system together with the span.

VLM-Rec measures each component of a recommendation cascade instead of assuming its value, and the same system serves as a controlled laboratory for a question the field is asking now. Finite catalogs make exact top-K ranking of a generative retriever computable; with that tool the dense-to-generative gap becomes a ledger of measured stage contrasts, and the ledger points practitioners to the tokenizer and to the head-and-objective step, measured at matched capacity.

A Key parameters

Table A1. Key parameters of every stage. All models train with Adam. The Beauty run uses the same values except where noted.

Component	Setting
Data	
Filtering and split	earliest interaction per user-item pair; 5-core on users and items; temporal leave-last-out (last item test, second-to-last validation)
Cold slice	test users whose target item is in the bottom quartile of training popularity

Component	Setting
Item encoders (frozen)	
Text	all-MiniLM-L6-v2 over title, description and features (first 1,000 characters); 384-d
Image	CLIP ViT-B/32 (LAION-2B weights) on the main product image; 512-d, plus a 1-d image-present flag
VLM profile	Qwen2.5-VL-7B-Instruct (3B on Beauty), greedy decoding, at most 320 new tokens, images capped at 768 × 768 pixels; profile text encoded by MiniLM, 384-d
Retrieval	
Two-tower	each tower an MLP 1,281 → 256 → 128 with LayerNorm, GELU and dropout 0.1; outputs L2-normalized
Training	in-batch softmax with log-frequency correction, temperature 0.05; batch 4,096; learning rate 0.002; 10 epochs
Hybrid mode inputs	item-ID embedding 64-d; price in 16 equal-frequency buckets, 8-d; category leaf (at least 20 items), 16-d
Index	exact inner product by default; HNSW with 32 links per node and search depth 256; IVF with 256 lists, 8 probed
Candidates	200 per request (264 fetched, then items the user has seen are removed)
Ranking and cascade	
Ranker	DIN over the last 30 items; DCN-v2 with 2 cross layers; MMoE with 4 experts of width 128 and 2 tasks (click, rating ≥ 4); model width 128; ID, price and category embeddings 32, 8 and 16-d
Ranker training	4 sampled negatives per positive, half drawn from retrieval candidates in the cascade variants; batch 2,048; learning rate 0.001; 3 epochs; GAUC on 100 sampled negatives per user
Pre-ranker	ranker without DIN and cross layers, distilled from the ranker's click logits for 2 epochs; keeps 50 of 200 candidates
Post-processing	score P(click) · P(satisfaction), blended with the retrieval score by a weight tuned on validation; MMR weight 0.7 or greedy DPP; final list of 20 by default
Semantic IDs	
RQ-VAE	encoder 1,281 → 512 → 256 → 64; 3 levels of 256 codes; commitment weight 0.25; codebook moving-average decay 0.99; unused codes reset after 200 steps; batch 2,048; learning rate 0.001; 300 epochs
TIGER-style retriever	decoder-only transformer, width 128, 4 layers, 4 heads; history of 20 items (60 tokens); vocabulary 769; beam width 20; batch 256; learning rate 0.001; 5 epochs
Loss-ledger study (Section 5)	

Component	Setting
Matched heads	shared transformer encoder as above; 999,168 (contrastive, temperature 0.05) and 999,041 (autoregressive) parameters
Training	up to 10 epochs with patience 3; batch 1,024; learning rate 0.001; epoch chosen by validation Recall@10 on a fixed sample of 2,048 users
Capacity arms	decoder widths 64, 128, 256, 512 (0.4M, 1.0M, 3.6M, 13.4M parameters), both heads retrained at every width
Statistics	3 seeds per catalog; 2,000 hierarchical seed × user bootstrap replicates; equivalence margin 10% of the local gap; stability rule 15 percentage points

B Reproduction

Every stage of the system is one `make` target or CLI command, configured by `configs/config.yaml`; `configs/scale.yaml` is the Beauty profile.

```
uv sync --all-extras    # every stage, incl. FAISS and MLflow
make data               # download, 5-core, split, embeddings
make retrieval          # two-tower variants + retrieval ablation
make ranking            # DIN + DCN-v2 + MMoE + ranking ablation
make rerank             # consistency grid, distillation, diversity
make vlm                # Qwen2.5-VL profiles + five-way ablation
make sid                # RQ-VAE semantic IDs + SID-vs-ID ablation
make serve              # API on :8000, Streamlit demo on :8501
uv run vlmrec <stage> --config configs/scale.yaml    # Beauty
```

System experiments ran on a single RTX 4090 (24 GB) under WSL2, and serving latencies are CPU-only. The loss-ledger study used about 118 GPU-hours across three catalogs, three seeds and the intervention runs. Numbers in Sections 2 to 4 come from the metrics files these commands write, and `docs/RESULTS.md` records the provenance of each table.

References

1. S. Rajput et al. Recommender systems with generative retrieval. *NeurIPS*, 2023.
2. Y. Hou et al. Bridging language and items for retrieval and recommendation. arXiv preprint, 2024.
3. N. Reimers and I. Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *EMNLP*, 2019.
4. A. Radford et al. Learning transferable visual models from natural language supervision. *ICML*, 2021.
5. S. Bai et al. Qwen2.5-VL technical report. arXiv preprint, 2025.
6. X. Yi et al. Sampling-bias-corrected neural modeling for large corpus item recommendations. *RecSys*, 2019.

7. J. Johnson, M. Douze and H. Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 2019.
8. Y. Malkov and D. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE TPAMI*, 2018.
9. G. Zhou et al. Deep interest network for click-through rate prediction. *KDD*, 2018.
10. R. Wang et al. DCN V2: Improved deep & cross network and practical lessons for web-scale learning to rank systems. *WWW*, 2021.
11. J. Ma et al. Modeling task relationships in multi-task learning with multi-gate mixture-of-experts. *KDD*, 2018.
12. J. Carbonell and J. Goldstein. The use of MMR, diversity-based reranking for reordering documents and producing summaries. *SIGIR*, 1998.
13. L. Chen, G. Zhang and H. Zhou. Fast greedy MAP inference for determinantal point process to improve recommendation diversity. *NeurIPS*, 2018.
14. D. Lee et al. Autoregressive image generation using residual quantization. *CVPR*, 2022.
15. J.-T. Huang et al. Embedding-based retrieval in Facebook search. *KDD*, 2020.